

Sistemas Distribuídos

Chamada de Procedimento Remoto (RPC)

André Luis Martinotto

UCS



UNIVERSIDADE DE CAXIAS DO SUL

Introdução

- Necessidade de desenvolver aplicações distribuídas de forma simples:
 - Solução: extender modelos de programação “convencionais”
- Três modelos:
 - Procedural: chamada remota de procedimentos (RPC)
 - Orientado a objetos: invocação remota de métodos (RMI)
 - Baseado em eventos: atendimento de eventos remotos

RPC - Remote Procedure Call

- Generalização do conceito de subrotinas:
 - Permite um programa chamar um procedimento que não se encontra no espaço de endereçamento do processo
- Permite a um cliente executar uma rotina em um servidor remoto:
 - Facilita o entendimento e desenvolvimento de aplicações do tipo cliente-servidor
 - Toda a comunicação entre os processos é similar a chamada de procedimentos convencional.

Modelo Básico do RPC

- A idéia do RPC é fazer em que a chamada remota de um procedimento se pareça com uma chamada local
 - Quem chama um procedimento remoto não deve ter que se preocupar se o procedimento é local ou não
 - O envio de mensagem de cliente para o servidor corresponde a chamada de um procedimento remoto
 - A resposta do servidor ao cliente corresponde ao retorno do procedimento remoto
- A forma como os dados são trocados entre as máquinas não é visível ao programador

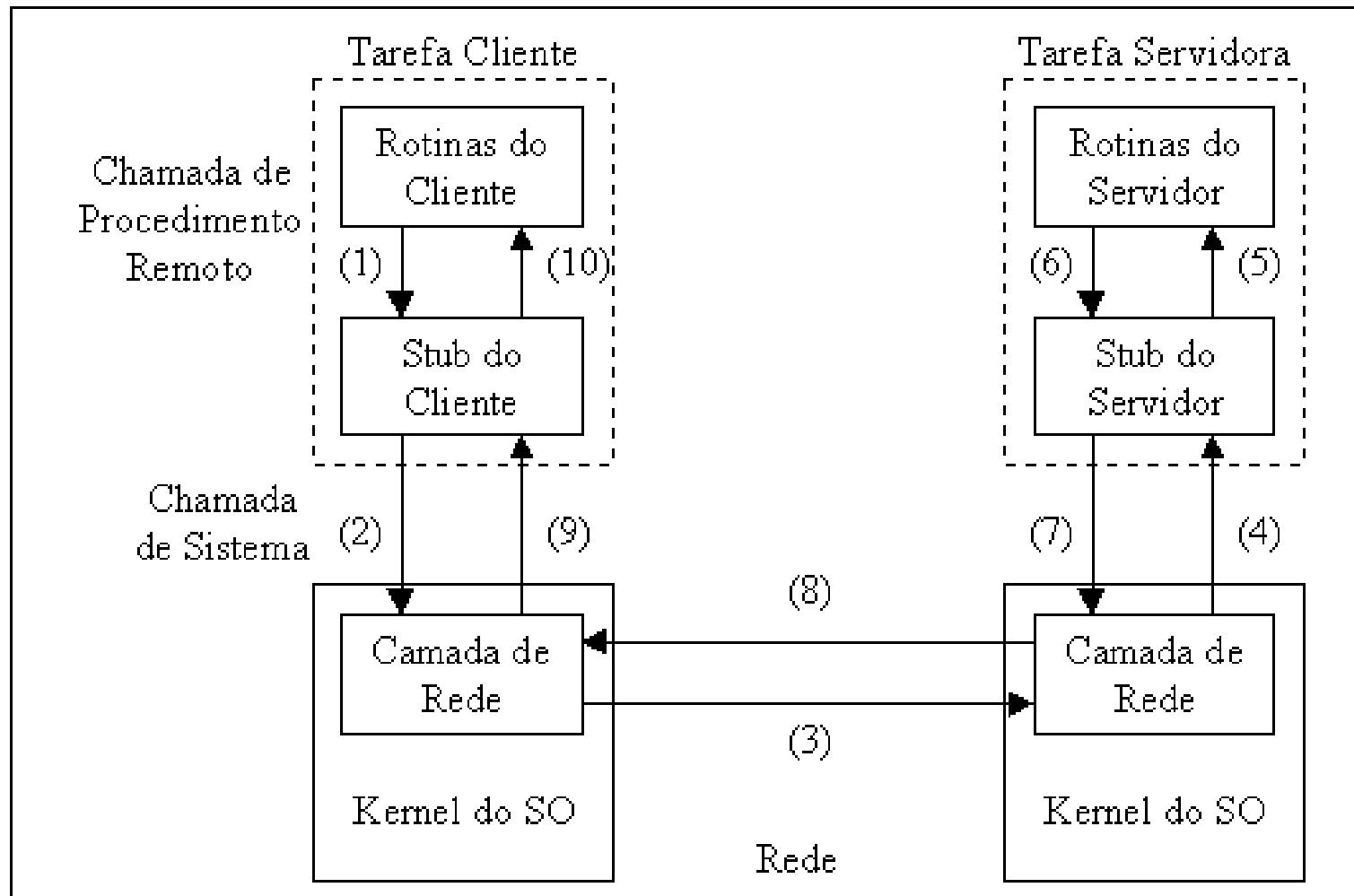
Modelo Básico do RPC

- A chamada é feita de maneira similar a uma chamada de local:
 - O processo rodando em uma máquina A chama um procedimento em uma máquina B
 - Informações são enviadas da A para a B através da passagem de parâmetros
 - O processo que chamou é suspenso
 - O procedimento localizado em B é executado
 - Informações de B para A podem ser retornadas no resultado do procedimento

Modelo Básico do RPC

- Apesar da chamada de função em RPC ser feita de forma similar a uma chamada local, o modo como ela é atendida difere bastante
- Em RPC a comunicação entre os processos é feita através de procedimentos especiais chamados stubs
 - Essas rotinas encontram-se nas tarefas cliente e servidora

Modelo Básico do RPC



Stubs

- Stubs: Implementações “falsas” dos procedimentos
- **Stub Cliente:**
 - Executa o marshalling dos argumentos
 - Envio de uma mensagem (requisição) ao processo servidor
 - Bloqueia para receber mensagem (resposta)
 - Efetua unmarshalling e repassa a aplicação
- **Stub Servidor:**
 - Executa o unmarshalling dos argumentos
 - Chama o procedimento associado (chamada local)
 - Executa o marshalling do resultado
 - Envia mensagem para o processo cliente

Identificação de Programas e Procedimentos

- Em um modelo RPC um procedimento remoto pode ser identificado por um par:
(program, version, procedure)
- **Program:**
 - Corresponde a um inteiro de 32 bits e identifica o programa remoto (único para cada programa)

Identificação de Programas e Procedimentos

- Numeração de programas RPC
 - Dividiu-se as faixas de números válidos em 8 grupos

0x00000000 – 0x1ffffff	Sun Microsystems, Inc
0x20000000 – 0x3ffffff	Usuários
0x40000000 – 0x5ffffff	Transitório (temporariamente)
0x60000000 – 0x7ffffff	Reservado
0x80000000 – 0x9ffffff	Reservado
0xa0000000 – 0xbffffff	Reservado
0xc0000000 – 0xdffffff	Reservado
0xe0000000 – 0xffffffff	Reservado

Identificação de Programas e Procedimentos

- **Version:**
 - Controle de versões para cada programa remoto.
 - Permite um computador rodar diferentes versões de um programa remoto.
 - Facilitando a migração de versões.
- **Procedure:**
 - Identificador do procedimento remoto.
 - Numerados seqüencialmente de 1,2...N.

Mapeamento de Porta

- Para tornar possível a comunicação entre cliente e servidor é necessário que cada serviço tenha uma porta e que essa seja conhecida
- Um programa RPC não usa uma porta conhecida:
 - Na inicialização o programa solicita ao sistema operacional uma porta não utilizada
 - Os clientes não podem acessá-lo diretamente

Mapeamento de Porta

- O sistema operacional pode escolher uma nova porta a cada vez que o programa remoto é inicializado:
 - O cliente não tem como conhecer a porta obtida pelo servidor
- **Solução:**
 - Máquinas que rodam um servidor remoto RPC provêem um mecanismo para que os clientes possam descobrir a porta de determinado programa remoto.

Mapeamento de Porta

- Esse mecanismo é chamado de RPC **port mapper**:
 - O servidor RPC ao ser inicializado recebe do Sistema Operacional o número da porta
 - Após o servidor registra no port mapper o número do programa RPC, a versão e a porta recebida
 - O cliente o port mapper enviando o número do programa RPC e a versão do mesmo
 - O port mapper retorna ao cliente o número da porta que esse programa remoto esta usando
 - O cliente sempre encontra o port mapper pois esse roda em uma porta conhecida (porta 111)

Passagem de Parâmetros

- **Passagem por Valor:**
 - Parâmetros são copiados para a mensagem e enviados ao servidor
 - Não representa maiores problemas
- **Passagem por Referência:**
 - Sem sentido, em princípio, por não haver espaço de endereçamento comum
 - **Soluções possíveis:**
 - Empregar uma passagem de parâmetros do tipo call-by-copy/restore
 - Usar DSM (Distributed Shared Memory)

Passagem de Parâmetros

- **Limitações - Exemplo rpcgen da SUN**
 - Limitação à passagem de parâmetros por valor.
 - Limitam ainda a único parâmetro de entrada.
 - A solução consiste em utilizar uma estrutura (uma struct em C) para a passagem de parâmetros.
- Restrições antes inexistentes na programação convencional podem dificultar o desenvolvimento de aplicações distribuídas

Representação dos Dados

- Enquanto as máquinas cliente e servidora forem idênticas o modelo RPC funciona bem.
- Em um grande sistema distribuído é comum a existência de vários tipos de máquinas
 - Cada uma delas com sua própria representação para números, caracteres e outros dados.

Representação dos Dados

- Essas diferenças podem gerar empecilhos inviabilizando uma comunicação entre os sistemas:
 - Uma solução consiste em utilizar uma representação independente de arquitetura (comumente XDR).
 - Sempre que for estabelecida a comunicação os dados serão convertidos neste tipo de representação.

Representação dos Dados

- Nesta solução não é possível identificar se ambos sistemas possuem a mesma representação interna;
 - Não sendo possível identificar se as duas etapas de conversão são desnecessárias
- Segundo John Corbin, o overhead gerado pela conversão é desprezível em aplicações de rede:
 - O tempo para atravessar as camadas de protocolos presentes nos clientes e servidores e transferir os dados pela rede torna os gastos com a conversão insignificantes

Desenvolvendo uma aplicação RPC

- O desenvolvimento de aplicação RPC sem o uso de bibliotecas ou ferramentas automáticas:
 - Complicado e consome muito tempo
- A implementação Sun RPC permite a geração dos stubs a partir de um arquivo de especificação:
 - O compilador **rpcgen** gera a partir de um arquivo de especificação;

Desenvolvendo uma aplicação RPC

- O arquivo de especificação é escrito em uma linguagem própria chamada IDL (Interface Definition Language):
 - O arquivo de especificação possui a extensão .x
- O rpcgen apresenta uma opção para geração de exemplos do código do servidor e do cliente

Desenvolvendo uma aplicação RPC

- **O arquivo de especificação contém:**
 - Declaração de constantes, dados globais e definição dos procedimentos remotos disponíveis (argumentos de entrada e saída);
 - O número do programa remoto, o número da versão e a definição dos procedimentos;
 - A definição de um procedimento consiste em um nome para cada procedimento e um número.

IDL - Hello

```
program helloprogram{  
    version helloversion{  
        string hello() = 1;  
    }=1;  
}=0x20000000;
```

Desenvolvendo uma aplicação RPC

- Os procedimentos remotos só podem ter um argumento e retornar um único resultados
 - Quando necessitam receber ou retornar mais que um valor é necessário o uso de estruturas;
- Não permite a passagem de valores por referência

Compilação do arquivo IDL

- A compilação do arquivo de IDL pode ser feita através do comando:

```
rpcgen -a file.x
```

Compilação do arquivo IDL

- São gerados os seguintes arquivos:

file_clnt.c	Código do stub do cliente (não alterar)
file_svc.c	Código do stub do servidor (não alterar)
file.h	Arquivo cabeçalho (não alterar)
Makefile.file	Arquivo para compilação (comando make)
file_client.c	Código de exemplo do cliente (aqui é inserido o código do cliente)
file_server.c	Código de exemplo do servidor (aqui é inserido o código do servidor)

Servidor - Hello

```
#include "hello.h"
#include <string.h>

#define TAM 50

char **
hello_1_svc(void *argp, struct svc_req *rqstp)
{
    static char * result;
    result = (char *) malloc (TAM * sizeof(char));
    strcpy(result,"RPC: Hello World");
    return &result;
}
```

Cliente - Hello

```
#ifndef DEBUG
    clnt = clnt_create (host, helloprogram, helloversion, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif /* DEBUG */

    result_1 = hello_1((void*)&hello_1_arg, clnt);
    if (result_1 == (char **) NULL) {
        clnt_perror (clnt, "call failed");
    }
    printf("%s\n",*result_1);
```

Compilação e Execução

- Para compilação e execução:
 - Renomear o arquivo Makefile.file para Makefile (mv Makefile.file Makefile)
 - Executar o comando make (make)
 - Rodar o servidor (./hello_server)
 - Rodar o cliente (./hello_client localhost)

IDL - Calculadora

```
struct args{  
    float op1;  
    float op2;  
};  
  
program calcprogram{  
    version calcversion{  
        float soma (struct args)=1;  
        float subtracao(struct args)=2;  
        float divisao (struct args)=3;  
        float multiplicacao(struct args)=4;  
    }=1;  
}=0x21000000;
```

Servidor - Calculadora

```
#include "calc.h"
```

```
float * soma_1_svc(struct args *argp, struct svc_req *rqstp){  
    static float result;  
    result = argp->op1 + argp->op2;  
    return &result;  
}
```

```
float * subtracao_1_svc(struct args *argp, struct svc_req *rqstp){  
    static float result;  
    result = argp->op1 - argp->op2;  
    return &result;  
}
```

...

Cliente - Calculadora

```
#ifndef DEBUG
```

```
    clnt = clnt_create (host, calcprogram, calcversion, "udp");
```

```
    if (clnt == NULL) {
```

```
        clnt_pcreateerror (host);
```

```
        exit (1);
```

```
    }
```

```
#endif /* DEBUG */
```

```
    result_1 = soma_1(&soma_1_arg, clnt);
```

```
    if (result_1 == (float *) NULL) {
```

```
        clnt_perror (clnt, "call failed");
```

```
    }
```

```
    result_2 = subtracao_1(&subtracao_1_arg, clnt);
```

```
    if (result_2 == (float *) NULL) {
```

```
        clnt_perror (clnt, "call failed");
```

```
    }
```